DataCraft · Intermediate Track

Capstone Project

FreshCart: build the data platform of a real-time grocery-delivery company

---

Project dates: Tuesday 13 October to Monday 26 October 2026 (two weeks)

Technologies: Docker · PostgreSQL · Python · dbt · Airflow

Cost: free. Everything runs on your laptop.

Contents

1. The scenario
2. The company's data sources
3. The pipeline you build (with the pipeline drawing)
4. What you must build
5. Every day: operations
6. Timeline
7. Deliverables and final demo
8. Grading
9. Rules
10. Getting started (step-by-step instructions)

1. The scenario

You have just joined FreshCart as its data engineering team. FreshCart delivers groceries in under an hour from 6 stores: Cairo (Maadi, Nasr City), Giza (Sheikh Zayed), Alexandria (Smouha), Riyadh (Olaya) and Dubai (Marina). It has been in business for two years: about 3 million orders so far, and around 5,000 new orders every day.

Management has no reliable numbers. Finance runs slow reports directly on the production database, and nobody can answer simple questions like "how much did each store earn yesterday, in US dollars?". Your job is to build the company's data platform.

> **This company is alive.** Its data is created in real time, on the Cairo clock: while the company is running, an order placed at 14:35 appears in the database at 14:35. If your laptop is off, the company catches up as soon as you restart it: the missed orders appear at once, with their real times. Every day there is new data and, as in any real company, things will change while you work.

2. The company's data sources

The starter kit runs FreshCart's systems on your laptop. You do not change them: you extract data from them, like a data engineer at a real company.

2.1 The production database (PostgreSQL)

	Connection
From your laptop	`localhost:5433`, database `freshcart_source`
From a container	`source-db:5432`
Your account	`analytics_reader` (read-only), password in .env: SOURCE_READER_PASSWORD

Tables in the schema `app`:

Table	Content
`stores`	store_id, store_name, city, country, currency (EGP, SAR, AED), opened_at
`customers`	customer_id, first_name, last_name, email, phone, city, signup_date
`products`	product_id, product_name, category, unit_price_usd (current catalogue price), is_active
`product_prices`	every catalogue price: product_id, unit_price_usd, valid_from
`orders`	order_id, customer_id, store_id, order_status (placed, delivered, cancelled), placed_at, delivered_at
`order_items`	order_item_id, order_id, product_id, quantity, unit_price, line_amount (store currency)
`payments`	payment_id, order_id, payment_method (card, wallet, cash), payment_status (paid, refunded), amount, paid_at

> Every table has `created_at` and `updated_at`. Rows change after they are created: an order goes from placed to delivered or cancelled, customers update their profile, prices change, refunds arrive days later.

2.2 Supplier stock files (CSV)

Two suppliers send an end-of-day stock file (units on the shelf at 22:00, per product and store) into the folder `source_files/suppliers/`:

Supplier	File	Arrives
FRESH (fruits & vegetables, dairy, bakery, meat & fish)	`fresh/fresh_stock_YYYY-MM-DD.csv`	every day at 22:00
GENERAL (everything else)	`general/GENERAL_STOCK_YYYYMMDD.csv`	every day at 23:30, but sometimes late (the next morning)

The two suppliers do not use the same format. Look at the files.

2.3 The exchange-rate API (HTTP)

FreshCart sells in EGP, SAR and AED; management reports in US dollars. The company subscribes to an external rate provider:

	Connection
From your laptop	`http://localhost:8000` (interactive docs: /docs)
From a container	`http://fx-api:8000`
API key	header `X-API-Key`, value in .env: FX_API_KEY

Endpoint	Returns
`GET /v1/rates?date=YYYY-MM-DD`	the rates valid on that day (units of currency per 1 USD)
`GET /v1/rates/history?start_date=&end_date=`	every published day of a period (max 31 days per call)
`GET /v1/currencies`	the currencies

It behaves like a real third-party provider: rates are published on business days (Sunday to Thursday) at 16:00 Cairo time; it limits how many calls you can make per minute; and it is not 100% reliable. Read its answers carefully.

2.4 Known data quality issues

Finding them is part of the job. Stakeholders have mentioned a few:

- "Our QA team creates test accounts in production."
- "Some customers seem to have two accounts."
- "Phone numbers are typed in every possible way."
- "One supplier's file is always a headache."

There are more.

3. The pipeline you build

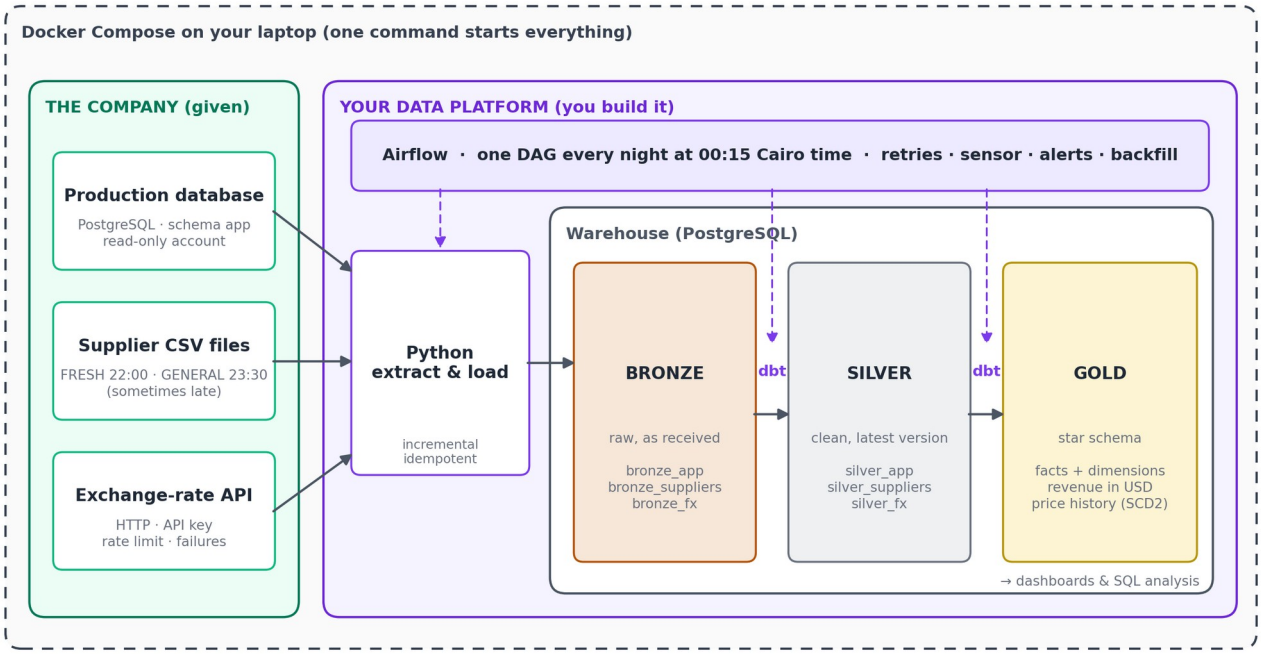


Figure 1. The FreshCart data pipeline

Reading the drawing from left to right:

1. **The company (given).** Three sources: the production database, the supplier CSV files and the exchange-rate API. Data arrives every day, in real time.
2. **Python extract & load.** Your scripts copy each source into the warehouse, one business day at a time, without changing anything.
3. **Bronze.** Raw data exactly as received, one schema per source system (bronze_app, bronze_suppliers, bronze_fx).
4. **Silver (built by dbt).** Clean, trustworthy tables, one schema per source: latest version of each row, standard formats, data problems flagged.
5. **Gold (built by dbt).** A star schema organised by business topic: facts and dimensions, revenue in USD, product price history.
6. **Airflow.** Runs the whole chain every night for the day that just ended: extract & load first, then dbt.
7. **Docker.** The company and your whole platform run as containers on your laptop, started with one command.

4. What you must build

4.1 Docker

- Your whole platform runs in containers, in the same `docker-compose.yml` as the company (add your services below the company's).
- One command starts everything. Passwords live in .env, never in your code.

4.2 The warehouse (PostgreSQL)

- A separate Postgres container: never run analytics on the production database.
- Layers as schemas: bronze and silver per source system, gold per business topic.

4.3 Extract & load (Python): sources to bronze

- **Initial full load:** all the history, once.
- **Then incremental,** one business day (Cairo time) at a time: only what was created or changed that day.
- **Idempotent:** loading the same day twice must never create duplicates.
- Bronze keeps the data as received (no cleaning), with where and when it was loaded.
- Supplier files: load each file once; late files must still be loaded.
- API: survive its failures and its limits.
- **Load a day only when the source has finished it:** after a restart, the company first catches up the missed hours, and a day loaded in the middle of that would be incomplete.

4.4 dbt: bronze to silver to gold

- **Silver:** one clean table per source table (latest version of each row, standard formats, flags for the problems you find).
- **Gold:** a star schema answering at least:
  - net revenue and cash collected per store and day, in USD and in local currency
  - number of orders, average basket, delivery time
  - customers (a person with two accounts counts once; test accounts excluded)
  - stockouts per store, category and supplier
  - the product price history (SCD type 2)
- **Tests:** generic tests on keys and relationships, at least two custom tests, at least one reconciliation test (two ways of computing the same number must agree), and source freshness.
- A description for every gold model.

4.5 Airflow

- One DAG that runs every night and processes the day that just ended: load the three sources, then build silver and gold.
- Dependencies between tasks, retries, a sensor for the supplier files, an alert when a task fails for good, and backfill (re-running past days).
- No business logic in the DAG file: the DAG orchestrates, your Python modules do the work.

4.6 SQL tuning

Finance's weekly report (`sql_tuning/legacy_finance_report.sql`) takes minutes on the production database. Make it fast and prove the results are identical. Details in `sql_tuning/README.md`.

5. Every day: operations

From your initial load onward, the company keeps producing data. Your pipeline must keep up every night until the end of the project. Every morning, check that last night's run succeeded and that the numbers make sense.

> **The company will change things without warning,** as real companies do: systems evolve, partners change their habits, bugs happen, services go down.

When something breaks, or worse, silently produces wrong numbers:

1. Detect it: tests and alerts are your eyes.

2. Understand the root cause.
3. Fix it in your platform, never by changing the company's systems.
4. Repair the affected days.
5. Write it down: what happened, the impact, what you changed.

For any incident where data was late or wrong, write a short postmortem using `docs/POSTMORTEM_TEMPLATE.md`.

6. Timeline

When	Goal	Checkpoint
Before day 1	Start the company (first start: about 25 minutes)	status shows today's orders
Week 1: 13-19 Oct	Warehouse, loaders (initial + daily), silver	Daily loads into bronze work; silver models tested
Week 2: 20-26 Oct	Gold, Airflow, tests, SQL tuning, documentation	The nightly DAG runs on its own; final demo

7. Deliverables and final demo

A GitHub repository with:

- your full platform: compose file, loaders, dbt project, DAG
- README.md: how to run it from zero, and an architecture diagram
- the sql_tuning deliverables (see its README)
- docs: one short note per change you had to handle, and your postmortem(s)

Final demo (15 minutes): architecture in 2 minutes; last night's run; the gold answers to the business questions; one incident you handled, from detection to fix; questions on your design choices.

8. Grading

Area	Weight
Docker & setup (one command, clean configuration)	10%
Extract & load (incremental, idempotent, robust)	15%
dbt silver (cleaning, tests)	15%
dbt gold (star schema, business answers, tests)	15%
Airflow (daily automation, retries, alerts, backfill)	15%
SQL tuning	15%
Handling the company's changes + postmortem	15%

9. Rules

- Free tools only.
- Do not modify the company's services (source-db, simulator, fx-api) or their settings in .env: everybody must have exactly the same company.

- AI assistants are allowed as tools, but you must be able to explain every line you submit: the demo questions will check.

10. Getting started

10.1 Requirements

- Docker Desktop, with at least 8 GB of memory for Docker (Docker Desktop > Settings > Resources) and about 20 GB of free disk
- VS Code (or any editor) and a SQL tool (DBeaver, pgAdmin, or VS Code's)
- A free GitHub account, for your project repository
- Windows: run all commands in PowerShell, inside the project folder

10.2 Start the company (before day 1: the first start takes about 25 minutes)

1. Unzip the starter kit and open a terminal in the `freshcart-capstone` folder.
2. Start the company:

```
docker compose up -d --build
```

3. Docker builds the company's programs, then creates two years of history (about 3 million orders). Follow it:

```
docker compose logs -f simulator
```

Ctrl+C stops watching; the company keeps running.

10.3 Check that the company is ready

```
docker compose exec simulator python -m simulator status
```

It is ready when:

- Last day started is today
- Latest order placed is a few minutes ago (Cairo time)
- there are about 3 million orders and about 720 + 720 supplier files

Run it again a few minutes later: the number of orders grows on its own.

10.4 Meet the three data sources

The production database: connect your SQL tool to localhost, port 5433, database freshcart_source, user analytics_reader, password reader. Or from the terminal:

```
docker compose exec source-db psql -U analytics_reader -d freshcart_source -c
"select * from app.orders order by placed_at desc limit 5"
```

The supplier files: open the folder source_files/suppliers/ (fresh/ and general/).

The exchange-rate API: open http://localhost:8000/docs, click Authorize, enter the key freshcart-fx-key, then try GET /v1/rates.

10.5 Build your platform

- Add your services (warehouse, loaders, dbt, Airflow) below the company's in `docker-compose.yml`. They reach the company by name: source-db:5432, fx-api:8000, and the supplier files through a volume on ./source_files/suppliers.
- Add your settings at the bottom of .env.

- Put your code in the project folder and push it to your GitHub repository.

10.6 Good to know

- The company only runs while Docker runs. If your laptop was off, it catches up on the next start: you get exactly the same data as everyone else. Give it a moment after a restart: status shows the current time once it has caught up.
- `docker compose down` stops everything and keeps the data.
- **Never run** `docker compose down -v` unless you want to delete the company's data (it would take about 25 minutes to recreate).
- A port is already used ("port is already allocated")? Another program uses 5433 or 8000: stop it, or tell your instructor.
- Do not modify the company's three services or their settings in .env.